Probeklausur: Mikrocomputertechnik 1

Bearbeitungszeit: 90 Minuten · Gesamtpunktzahl: 100 Punkte

Name: _____ **Matrikelnummer:** _____

Kurs / Studiengruppe: _____

Erlaubte Hilfsmittel: Das offizielle RISC-V Green Card / Reference Sheet (wird ausgeteilt).

Aufgabe	Thema	Punkte	erreicht
1	Zahlendarstellung	10	
2	Code-Analyse / Tracing	15	
3	C-to-Assembly	25	
4	Funktionen / Stack (Kurzfragen)	15	
5	Steuerwerk / Datenpfad	15	
6	Pipelining / Hazards	20	
	Summe	**100**	

Aufgabe 1: Zahlendarstellung (10 Punkte)

1. Wandeln Sie 85_{10} in eine 8-Bit-Binärzahl und in Hexadezimal um. (4 Pkt)
2. Bilden Sie das 8-Bit-Zweierkomplement von -85. (4 Pkt)
3. Welches Byte liegt in Little-Endian an Adresse `0x1000`, wenn das Wort `0xAABBCCDD` ab `0x1000` gespeichert wird? (2 Pkt)

Aufgabe 2: Code-Analyse / Tracing (15 Punkte)

Alle Register starten bei 0. Welchen dezimalen Wert enthalten `t0`, `t1` und `t2` nach Ausführung?

```
li t0, 5
li t1, 3
sub t2, t0, t1
slli t2, t2, 2
add t0, t0, t2
```

- `t0` = _____ (8 Pkt)
- `t1` = _____ (2 Pkt)
- `t2` = _____ (5 Pkt)

Aufgabe 3: C-to-Assembly (25 Punkte)

Übersetzen Sie den C-Code in RISC-V-Assembler. Nutzen Sie `s0` für `sum`, `t0` für `i`. Invertierte Abbruchbedingung (`bge` / `bgt`).

```
int sum = 0;
int i = 1;

while (i <= 10) {
    sum = sum + i;
    i++;
}
```

Ihre Assembler-Lösung

Aufgabe 4: Funktionen und Stack (15 Punkte)

Beantworten Sie kurz:

1. Welche Register übergibt der Caller typischerweise für die ersten beiden Argumente? (3 Pkt)
2. Ist `ra` Caller-saved oder Callee-saved — und warum muss eine Non-Leaf-Funktion `ra` sichern? (6 Pkt)
3. Skizzieren Sie den Prolog einer Funktion, die nur `s0` sichert. Der Frame ist 16 Byte groß (ILP32). (6 Pkt)

Aufgabe 5: Steuerwerk / Datenpfad (15 Punkte)

Füllen Sie die Wahrheitstabelle für `sub` und `lw` aus (0, 1 oder X). Signalnamen nach Patterson & Hennessy (Kurskonvention).

Instruktion	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch
`sub`						
`lw`						

Aufgabe 6: Pipelining und Hazards (20 Punkte)

5-stufige Pipeline mit Hazard Detection und Forwarding. Füllen Sie das Timing-Raster aus und markieren Sie Stalls.

```
lw t0, 0(a0)
addi t0, t0, 5
add t1, t0, t0
```

Befehl / Takt	1	2	3	4	5	6	7	8
`lw t0, 0(a0)`								
`addi t0, t0, 5`								
`add t1, t0, t0`								